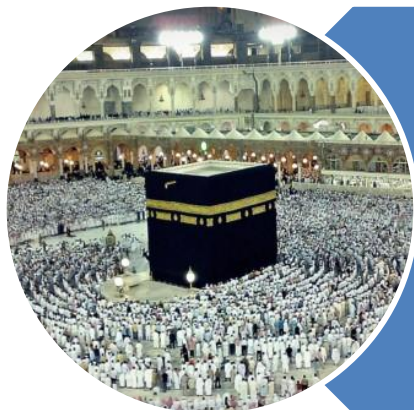


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Linked List

Operations

C

P

C

S

2

0

4

3

Operations

- Is Empty
- Traversing
- Modifying
- Searching
- Insertion
- Deletion
- Merging

Linked List

Operations

(Deletion)

C

P

C

S

2

0

4

5

Linked Lists - Deletion

- You must search for the node that you want to delete (**remember, we are using sorted lists**)
- If found, you must delete the node from the list
- This means that you change the various link pointers
 - The **predecessor** of the deleted node must point to the deleted nodes **successor**

C

P

C

S

2

0

4

6

Linked List – Deletion

- 4 – Cases
 - Delete the **only node** in the list
 - Delete the **first node**
 - Delete the **last node**
 - Anywhere **between** two nodes
- Pre – Condition
 - List must be **non - empty**

C

P

C

S

2

0

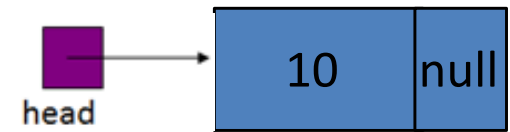
4

7

Linked List – Deletion (only node of list)

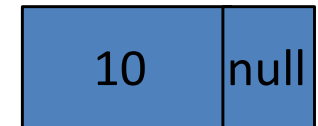
- Check if it is the only node

- `head.data == value &`
- `head.next == null`



- Make the node inaccessible

- `head = null` **OR**
- `head = head . next`



- **Java garbage** collector will remove the node

C

P

C

S

2

0

4

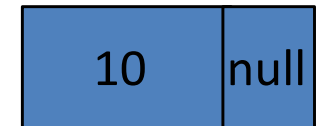
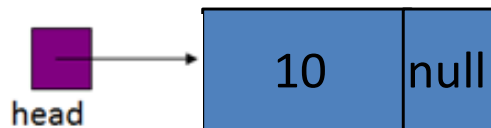
8

Linked List – Deletion (only node of list)

```
public void delete(int value) {
    if (!isEmpty()) {

        if (head.data == value && head.next == null)

            head = head.next;
    }
}
```



- **Value = 10**

C

P

C

S

2

0

4

Linked List – Deletion (first node)

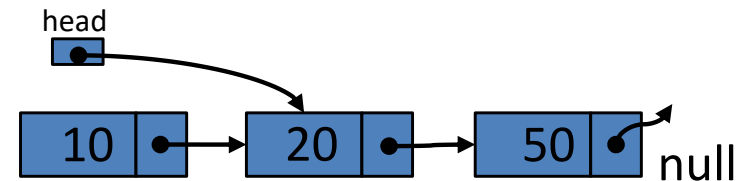
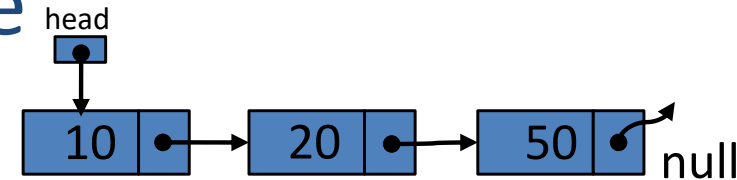
- Check if it is the first node

○ `head.data == value`

- Move head to point next node

○ `head = head . next`

- **Java garbage** collector will remove the first node



C

P

C

S

2

0

4

10

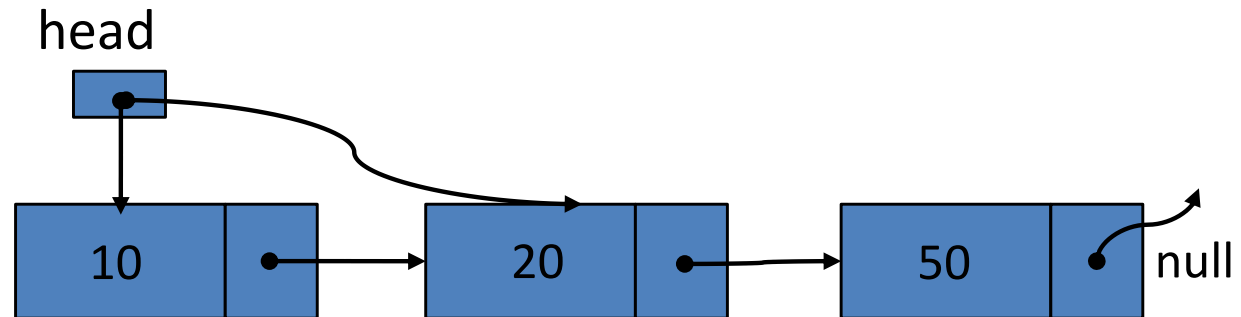
Linked List – Deletion (first node)

```
public void delete(int value) {
    if (!isEmpty()) {

        if (head.data == value)

            head = head.next;

    }
}
```



- **Value = 10**

"No Bonus" marks for this course anymore

C

P

C

S

2

0

4

11

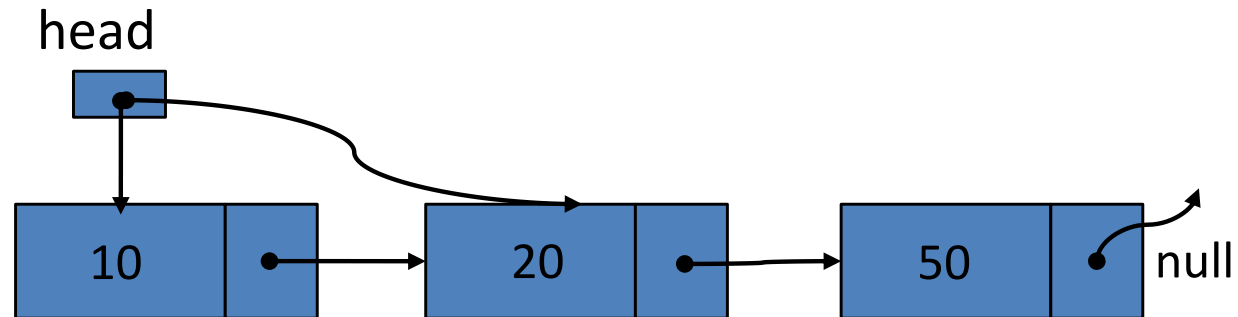
Linked List – Deletion (Only node or first node)

```
public void delete(int value) {
    if (!isEmpty()) {

        if (head.data == value)

            head = head.next;

    }
}
```

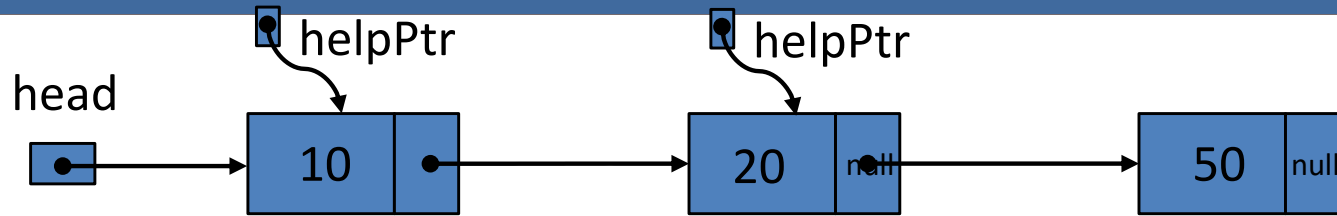


- **Value = 10**

“No Bonus” marks for this course anymore

C
P
C
S

2
0
4



- Traverse till 2nd last node
 - helpPtr = head
 - helpPtr.next.next != null
- Make helpPtr's next point to null
 - helpPtr.next = null
- **Java garbage** collector will remove the last node

Linked List – Deletion (last node)

“No Bonus” marks for this course anymore

C

P

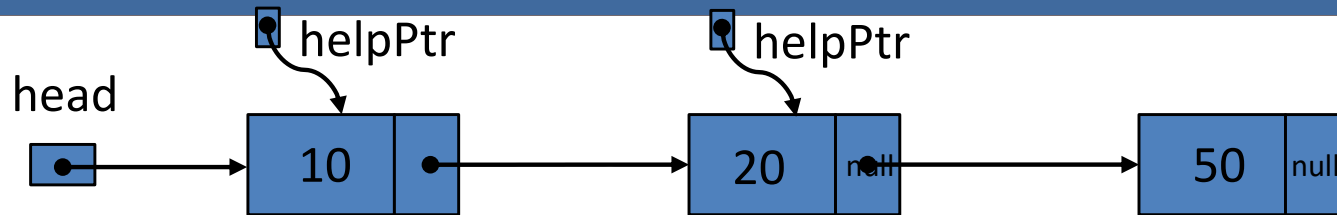
C

S

2

0

4



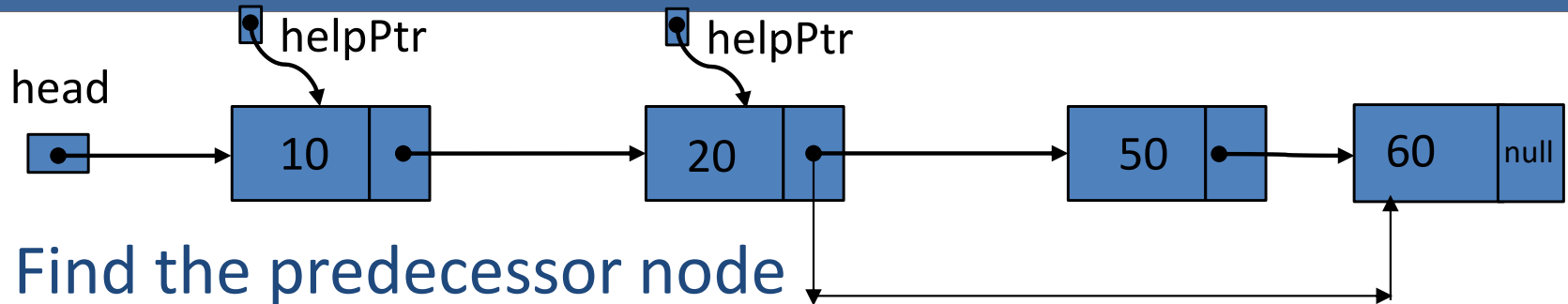
```

public void delete(int value) {
    if (!isEmpty()) {
        if (head.data == value)
            head = head.next;
        else{
            LLnode help_ptr = head;
            while (help_ptr.next.next != null) {
                help_ptr = help_ptr.next;
            }
            helpPtr.next = null;
        }
    }
}

```

Linked List – Deletion (last node) **50**

"No Bonus" marks for this course anymore



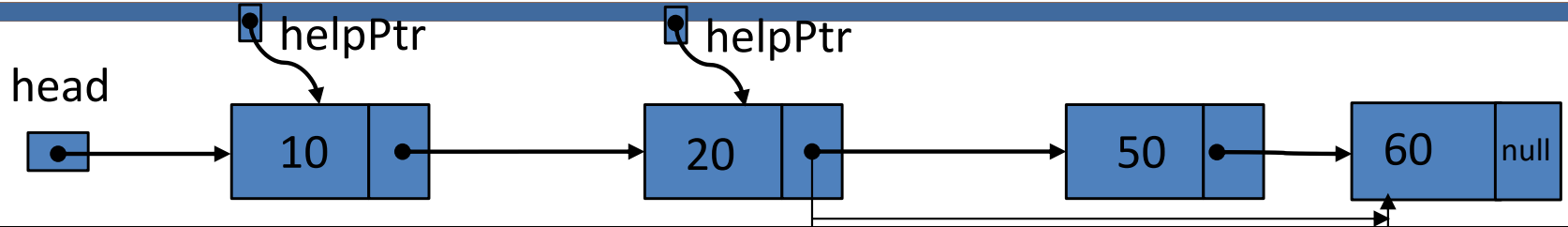
- Find the predecessor node
 - `helpPtr = head` (initialize)
 - `helpPtr.next != null` (loop condition)
 - `helpPtr.next.data == value` (finding value)
- Connect predecessor node with the successor node
 - `helpPtr.next = helpPtr.next.next`
- Java garbage** collector will remove the last node

Linked List – Deletion (in middle)

“No Bonus” marks for this course anymore

C
P
C
S

2
0
4



```
public void delete(int value) {
    if (!isEmpty()) {
        if (head.data == value)
            head = head.next;
        else {
            LLnode help_ptr = head;
            while (help_ptr.next != null) {
                if (help_ptr.next.data == value) {
                    help_ptr.next = help_ptr.next.next;
                    break;
                }
                help_ptr = help_ptr.next;
            }
        }
    }
}
```

Linked List – Deletion (in middle) **50**

"No Bonus" marks for this course anymore

C

P

C

S

2

0

4

Linked List – Deletion (Complete Code)

```
public void delete(int value) {
    if (!isEmpty()) {
        if (head.data == value) // if "value" is in the first node
            head = head.next;
        // ELSE, value is (possibly) elsewhere in the list
        else {
            LLnode help_ptr = head;
            while (help_ptr.next != null) {
                if (help_ptr.next.data == value) {
                    help_ptr.next = help_ptr.next.next;
                    break;
                }
                help_ptr = help_ptr.next;
            }
        }
    }
}
```


Linked List

Operations

(Merging)

C

P

C

S

2

0

4

Linked Lists - Merge

- Combining the data of two or more lists into any other data structure e.g. another linked list
- Find the **last node** of first linked list
- Connect the **last node** of **1st** list to the **first node** of **2nd** list
- Make a new pointer to merged list (**OPTIONAL**)

“No Bonus” marks for this course anymore

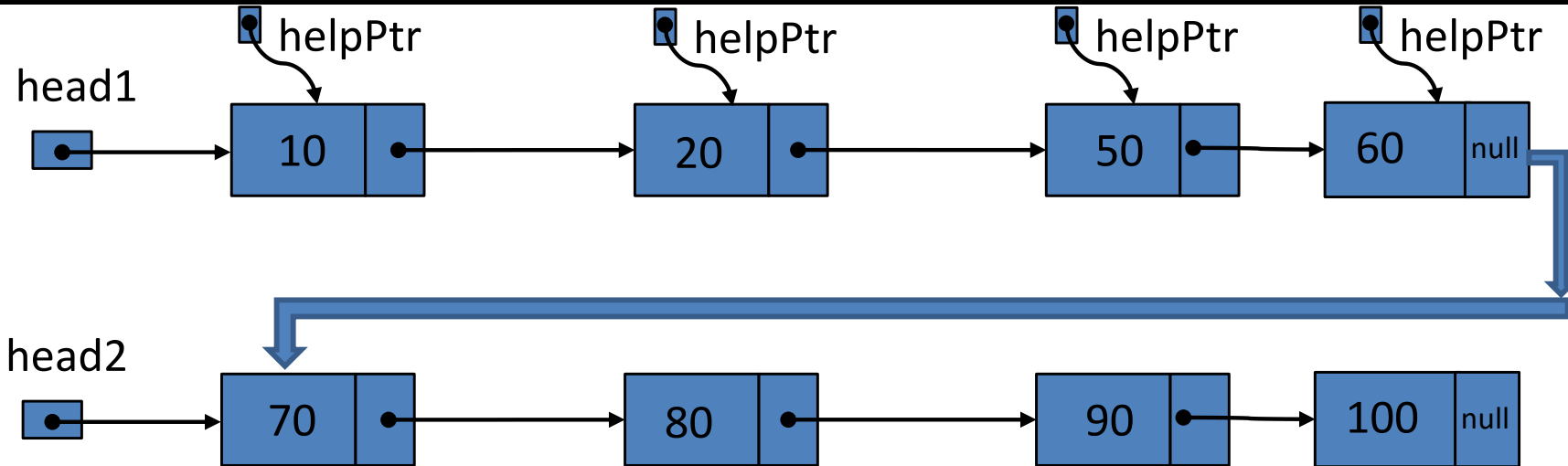
C
P
C
S

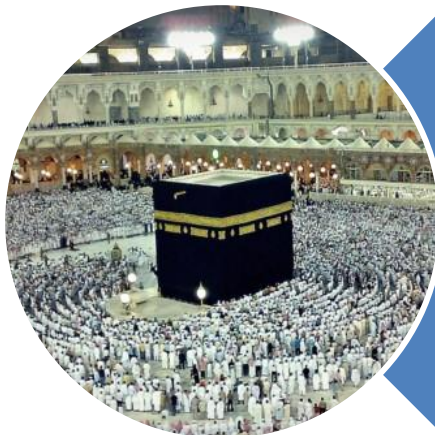
2
0
4

Linked List – Merge (Non Empty lists)

```
public LNode merge(LNode head1, LNode head2) {
    LNode help_ptr = head1;
    while (help_ptr.next != null) {
        help_ptr = help_ptr.next;
    }

    help_ptr.next = head2
    return head1;
}
```





Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**